
-->Algorithm:

--infix to postfix converter:--

```
Stack s
while(not end of input)
{
c=next input character;
if(c is an operand)
add c to postfix expression
else{
while(!s.empty() && prcd(s.top(),c))
{
operator=s.pop();
add operator to the postfix expression
}
if(s.empty() || symbol != '(')
{s.push(c);
}
else
{s.pop();}

}
while(!s.empty())
{operator=s.pop();
add operator to the postfix expression
}

}
```

Precedence Method:

- `prcd(op1,op2)` //return true if op1 precedence is higher than op2 otherwise return false
- `prcd(op,'(')` //false for all kind of operators
- `prcd(op,')')` //true for all kind of operators

- `prcd('(',op) // false`
 - `prcd('(',')') //false`
-

$((A+B)-C*(D/E))+F$

Expression= $AB+CDE/*-F+$

Postfix expression evaluation:

Algorithm:

Sack s;

while(not end of input)

{

e= next input element;

if(e is an operand)

{

s.push(e);

}else

{

Op2=s.pop();

Op1=s.pop();

Result=apply e on op1 and op2

s.push(Result);

}

}

finalResult=s.pop();

}